

Full Stack Development with MERN

Project Documentation format

HOUSEHUNT

Smart Real Estate Rental Platform Using MERN

Stack

Bachelor of Technology

In

Artificial Intelligence and Data Science

Submitted By

Lavanya Singamsetti

Roll No.: 23BQ1A54D2

e-mail: 23bq1a54d2@vvit.net

Full Stack Development with MERN

Project Documentation format

1. Introduction

1.1 Project Title

HouseHunt – Smart Real Estate Rental Platform

1.2 Team Members

Team Member: Lavanya Singamsetti

Role and Responsibilities:

- Requirement Analysis
- UI/UX Design
- Frontend Development using React.js
- Backend Development using Node.js & Express.js
- MongoDB Database Design
- REST API Development
- JWT Authentication Implementation
- Google Maps Integration
- Testing and Debugging
- Project Documentation

2. Project Overview

2.1 Purpose:

The primary purpose of HouseHunt is to simplify the property rental process by providing a centralized web platform where tenants can search for rental properties and property owners can efficiently manage their property listings. Traditional methods of finding rental properties involve brokers, newspaper advertisements, and manual communication, which consume time and increase costs. HouseHunt eliminates these challenges by providing a secure and digital platform that connects property owners and tenants directly.

The project aims to improve transparency, reduce dependency on intermediaries, and provide users with an efficient and reliable property management system. The application offers an intuitive interface, secure authentication, property search with filters, property reviews, visit scheduling, Google Maps integration, wishlist functionality, and owner dashboards for effective property management.

The application is developed using the MERN Stack to ensure scalability, maintainability, responsiveness, and high performance. It demonstrates modern web development practices by implementing RESTful APIs, JWT authentication, cloud database storage, and responsive frontend components.

Find Your Dream Home

Discover apartments, villas, and houses at the best locations with HouseHunt.

[Search](#)[Browse Properties](#)[Get Started](#)

2.2 Features:

HouseHunt offers numerous features to enhance the user experience and simplify property rental management.

2.2.1 User Registration

- Secure user registration
- Email validation
- Password encryption using bcrypt
- User data stored securely in MongoDB

Register

[Register](#)

Already have an account? [Login Here](#)

2.2.2 User Login

- Secure authentication using JWT
- Login validation
- Protected routes
- Session management



HomePropertiesDashboard

LoginRegister

Login

Login

New User? [Register Here](#)

2.2.3 Property Management

- Add Property
- Edit Property
- Delete Property
- Upload Property Images
- Property Description Management
- Price Management



HomePropertiesDashboardAdd Property

Welcome, LavanyaLogout

Clear Filters



₹ 50000/month

House

 Chennai

 4 Bedrooms

 5 Bathrooms

View Details



₹ 15000/month

hotel

 pondi

 1 Bedrooms

 2 Bathrooms

View Details



₹ 45000/month

villa

 tulsi

 2 Bedrooms

 2 Bathrooms

View Details



₹ 8000000/month

vv

 Vizag

 2 Bedrooms

 2 Bathrooms

View Details



localhost:5173/property/6a51c98fb9c0e1865c53bee7







2.2.4 Property Search

- Search by Location
- Search by Property Name
- Search by Price
- Real-time Filtering

Home Properties Dashboard Add PropertyWelcome, Lavanya Logout

2 BHK

Clear Filters



₹ 8000000/month

vv
Vizag
2 Bedrooms
2 Bathrooms

View Details

2.2.5 Property Details

- Property Images
- Owner Information
- Google Maps Integration
- Bedrooms
- Bathrooms
- Price
- Description

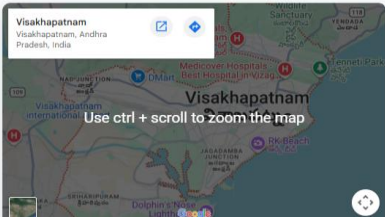
Home Properties Dashboard Add PropertyWelcome, Lavanya Logout



vv
₹ 8000000
Vizag
2 Bedrooms 2 Bathrooms

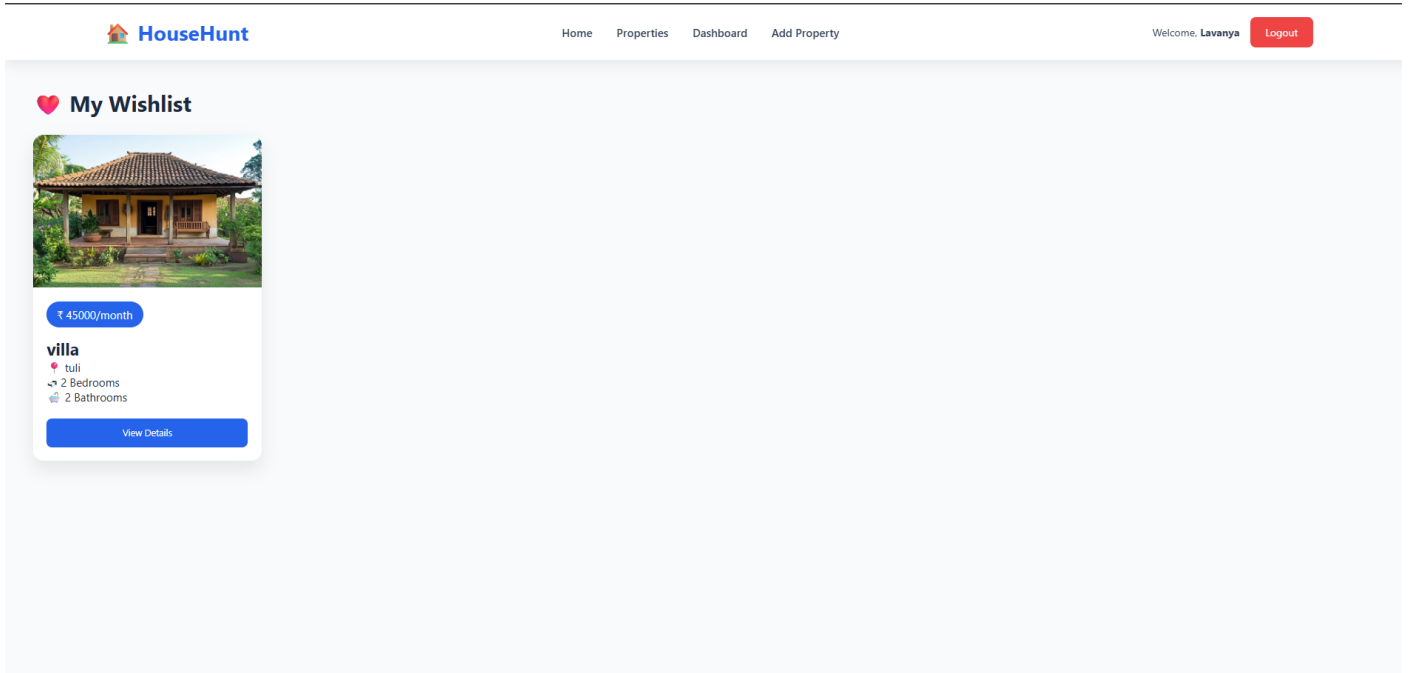
Description
it has good view

Listed By
Name: Singamsetti Lavanya
Email: lavanyasingamsetty1234@gmail.com

Property Location


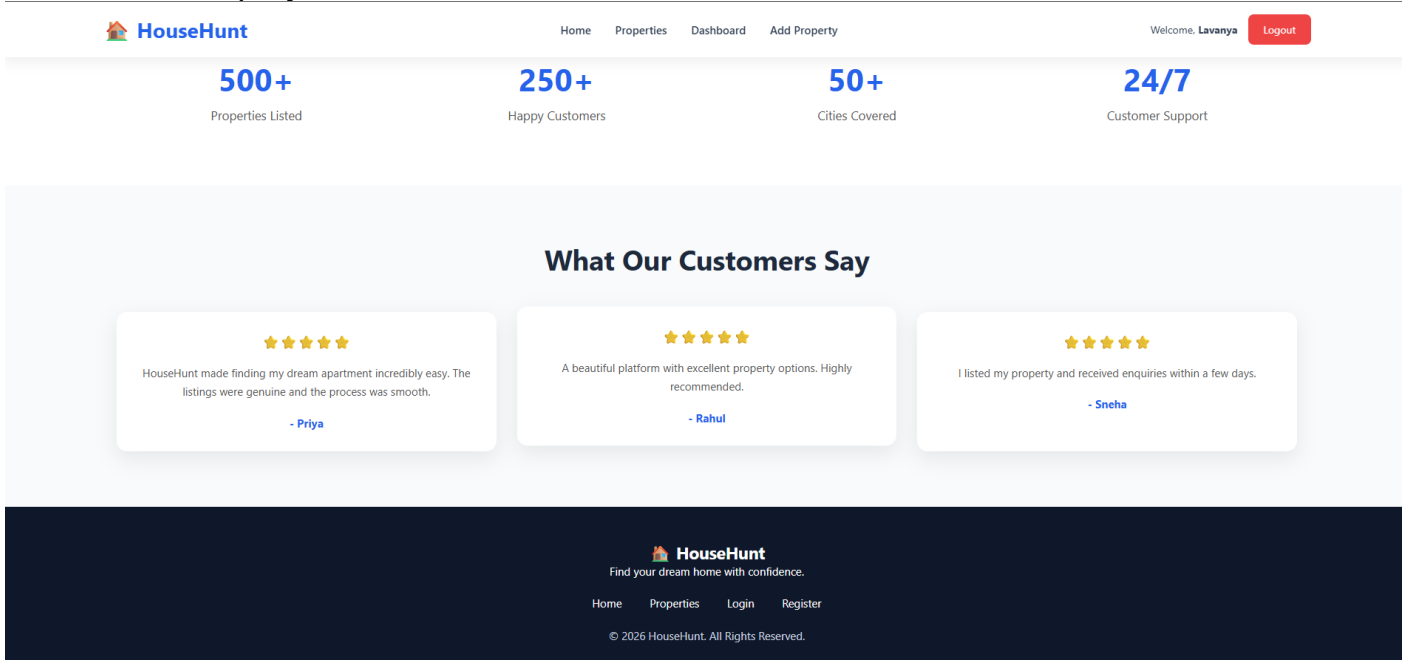
2.2.6 Wishlist

- Save Favorite Properties
- Remove Saved Properties
- Personalized Wishlist



2.2.7 Reviews and Ratings

- User Ratings
- Customer Reviews
- Property Feedback



2.2.8 Visit Scheduling

- Schedule Property Visit
- Select Date
- Select Time
- Owner Approval
- Accept/Reject Visit Request

 Home Properties Dashboard Add Property Welcome, Lavanya Logout

Schedule a Visit

dd-mm-yyyy

Select Time

Schedule Visit

Wishlist

Email Owner

WhatsApp

Reviews & Ratings

★★★★★

Write your review...

Submit Review

Ram

★★★★★

nice place to hang out with family

https://wa.me/918121178285

2.2.9 Dashboard

- Property Statistics
- Visit Requests
- Property Management
- Quick Actions
- Owner Profile

 Home Properties Dashboard Add Property Welcome, Lavanya Logout

Welcome, Lavanya

Manage your properties from one place.

1My Properties

0Visit Requests

0Saved Homes

0Messages

My Properties



₹ 50000/month

House

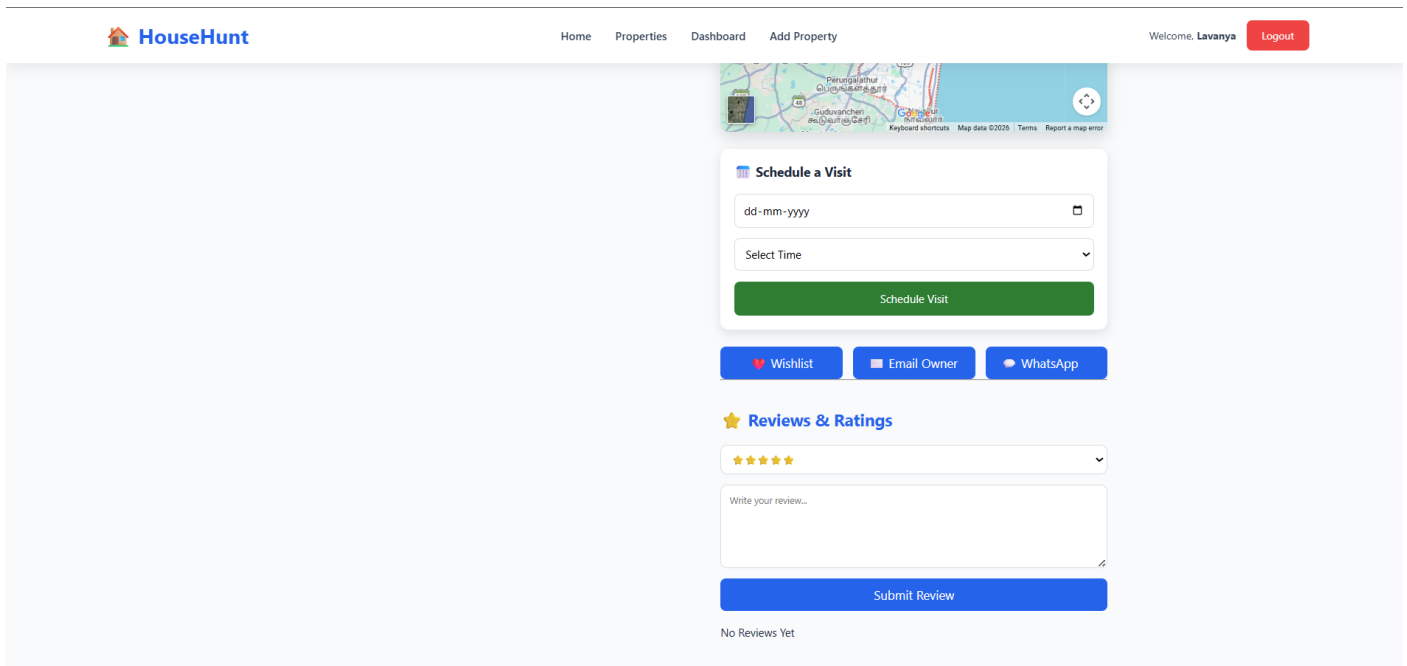
Chennai

4 Bedrooms

5 Bathrooms

2.2.10 Contact Owner

- Email Integration
- WhatsApp Integration
- Direct Communication



3. Architecture

3.1 Frontend:

The frontend of HouseHunt is developed using React.js, which enables the creation of reusable, responsive, and interactive user interfaces. React Router is used for client-side routing, allowing seamless navigation between pages without refreshing the browser. Axios is used to communicate with the backend REST APIs. CSS is used to design responsive layouts that work across desktops, tablets, and mobile devices.

The frontend consists of multiple reusable components such as PropertyCard, Navbar, Dashboard Cards, Review Section, Visit Scheduler, Wishlist Components, and Authentication Pages.

State management is handled using React Hooks such as useState and useEffect, ensuring efficient rendering and improved user experience.

3.2 The backend is developed using Node.js and Express.js, which provide a lightweight, scalable, and efficient server environment.

The backend performs the following operations:

- User Authentication
- Property CRUD Operations
- Review Management
- Wishlist Management
- Visit Scheduling
- Dashboard Statistics
- JWT Token Verification
- Authorization Middleware
- Error Handling
- Image Upload using Multer

RESTful APIs facilitate communication between the frontend and backend.

3.3 Database:

MongoDB Atlas is used as the cloud-based NoSQL database for storing application data.

The database contains multiple collections:

- Users

- Properties
- Reviews
- Visits
- Wishlist

Relationships between collections are maintained using MongoDB Object IDs and Mongoose references.

The database ensures secure storage, scalability, and fast retrieval of information.

4.Setup Instructions

4.1 Prerequisites:

Before running the project, ensure the following software is installed:

- Node.js
- npm
- MongoDB Atlas Account
- Git
- Visual Studio Code
- Postman
- Google Chrome

4.2 Installation:

Step 1 - Clone the repository

```
git clone <repository-url>
```

Step 2 - Navigate to Client Folder

```
cd client
```

Step 3 - Install Dependencies

```
npm install
```

Step 4 - Navigate to Server Folder

```
cd ../server
```

Step 5 - Install Dependencies

```
npm install
```

Step 6 - Create Environment Variables

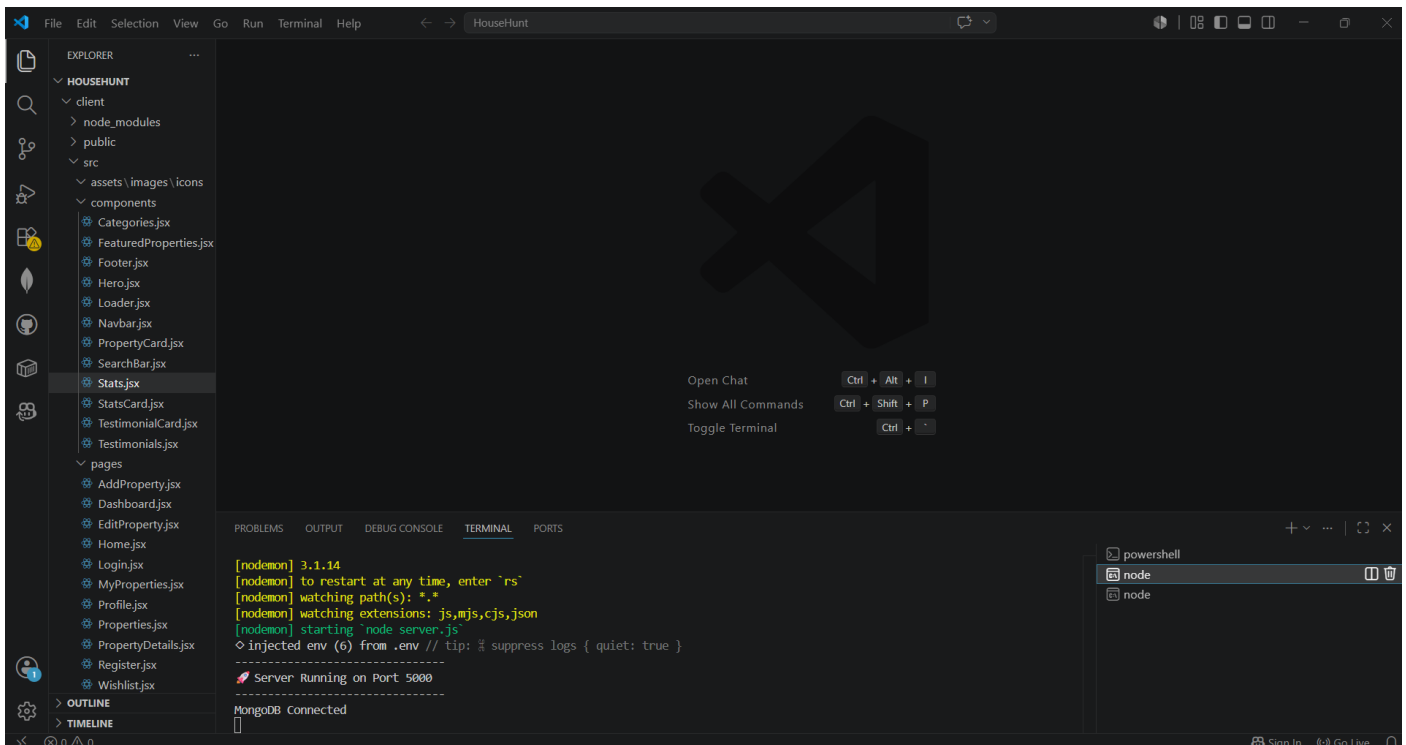
```
PORT=5000  
MONGO_URI=your_mongodb_connection  
JWT_SECRET=your_secret_key
```

Step 7 - Start Backend

```
npm run dev
```

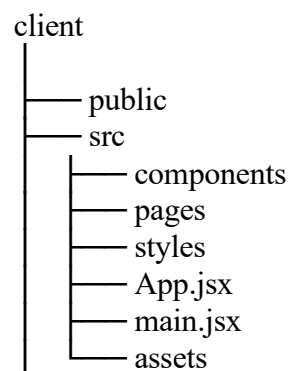
Step 8 - Start Frontend

```
npm run dev
```

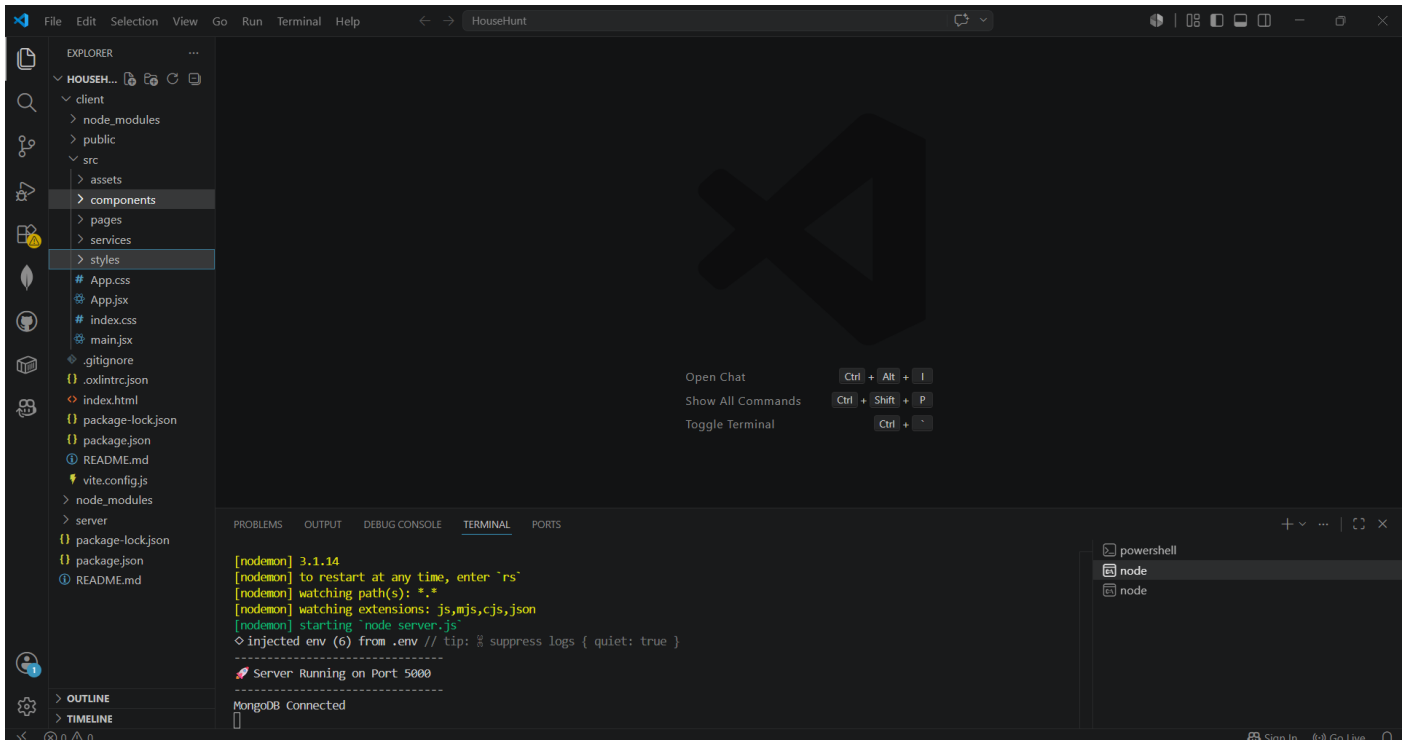


5.Folder Structure

5.1 Client: The client folder contains the complete React application.

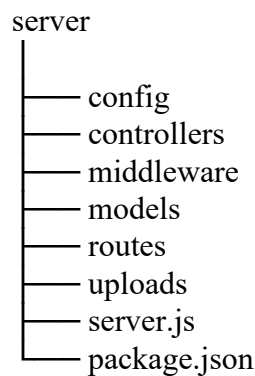


Each folder is organized to improve maintainability and code reusability.

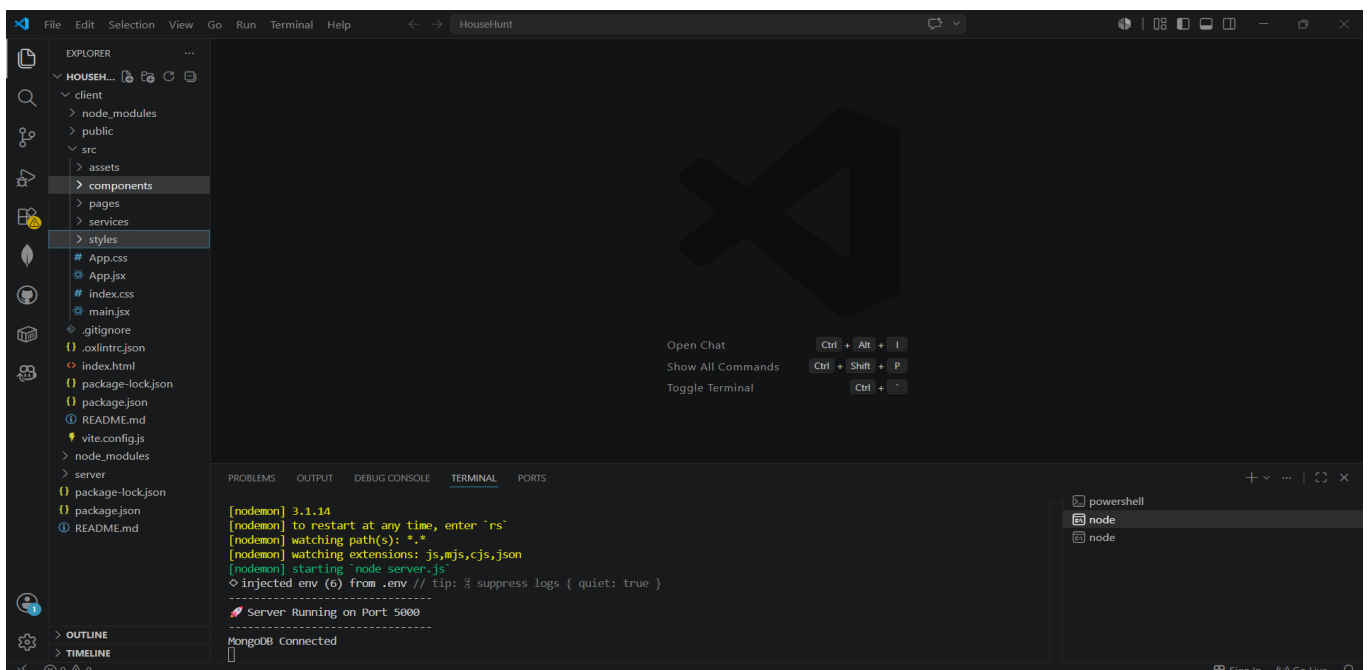


5.2 Server:

The server folder contains backend source code.



Each folder has a specific responsibility such as routing, business logic, middleware, and database operations.



6. Running the Application

6.1.1 Provide commands to start the frontend and backend servers locally.

6.1.2 Frontend

```
cd client
npm run dev
```

6.1.3 Backend

```
cd server
npm run dev
```

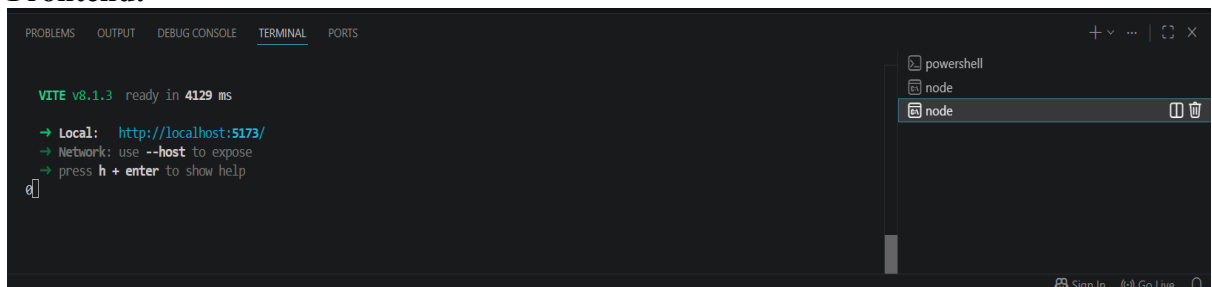
6.1.3 Once both servers are running:

Frontend: <http://localhost:5173>

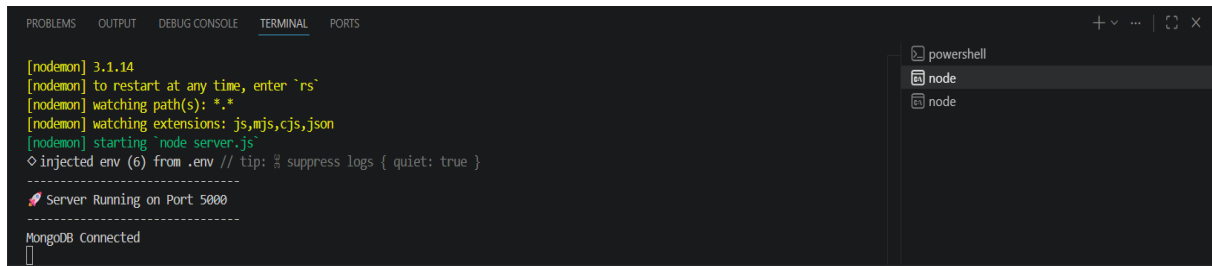
Backend: <http://localhost:5000>

Open the frontend URL in your browser to access the application.

Frontend:

A screenshot of a VS Code terminal window. The terminal shows the output of running 'npm run dev' in the client directory. It displays 'VITE v8.1.3 ready in 4129 ms' and provides instructions for local and network access. The local URL is 'http://localhost:5173/'. The network instruction says 'use --host to expose'. A prompt 'press h + enter to show help' is also visible. The terminal is titled 'TERMINAL' and has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'PORTS'. On the right side of the terminal, there is a sidebar showing the file explorer with 'powershell', 'node', and 'node' files.

Backend:

A screenshot of a VS Code terminal window. The terminal shows the output of running 'npm run dev' in the server directory. It displays '[nodemon] 3.1.14' and '[nodemon] to restart at any time, enter `rs`'. It also shows '[nodemon] watching path(s): *.*' and '[nodemon] watching extensions: js,mjs,cjs,json'. The terminal indicates 'starting `node server.js`' and 'injected env (6) from .env // tip: ⚙ suppress logs { quiet: true }'. A status bar at the bottom says 'Server Running on Port 5000'. The terminal is titled 'TERMINAL' and has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'PORTS'. On the right side of the terminal, there is a sidebar showing the file explorer with 'powershell', 'node', and 'node' files.

7. API Documentation

7.1 Document all endpoints exposed by the backend.

The backend exposes several REST APIs for communication between the client and the server.

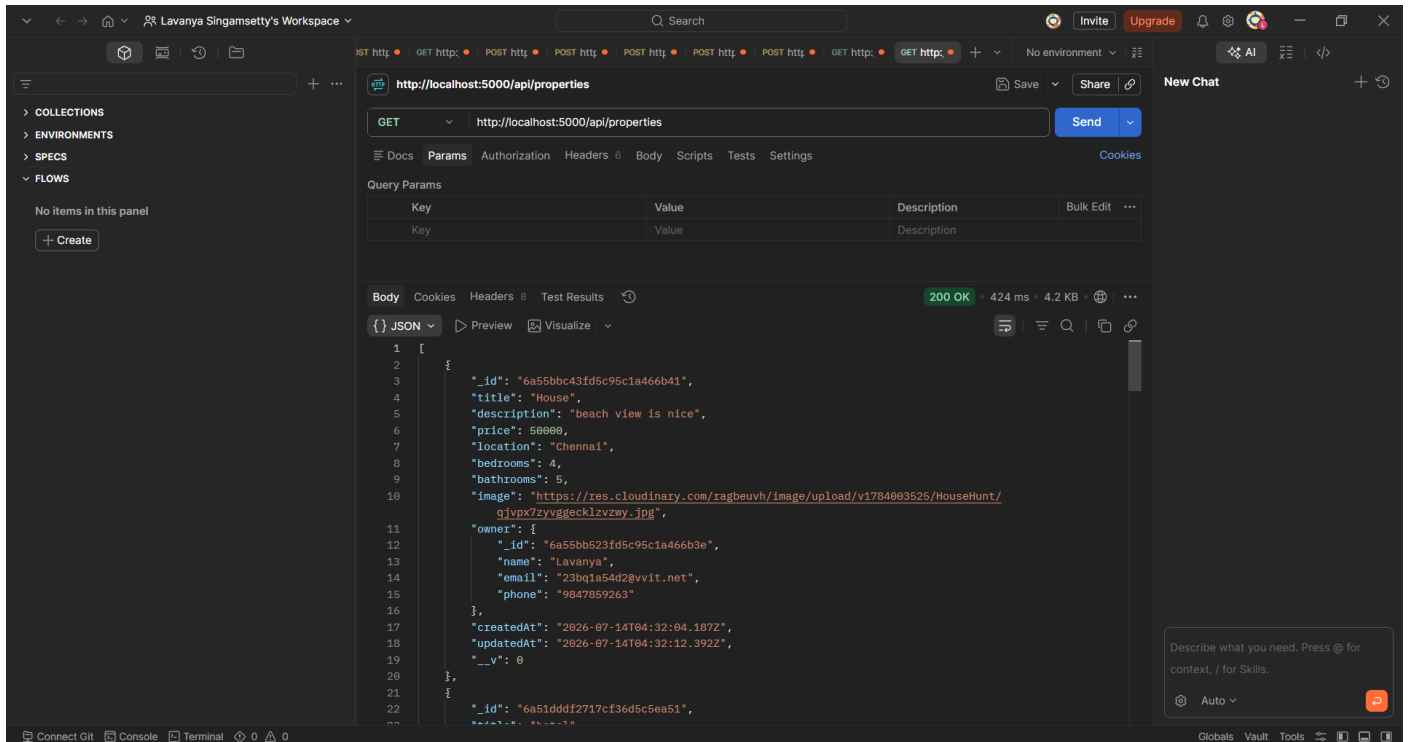
Examples include:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/properties
 - POST /api/properties
 - PUT /api/properties/:id
- DELETE /api/properties/:id
- POST /api/wishlist
- GET /api/wishlist
- POST /api/reviews
- GET /api/reviews/:id
- POST /api/visits
- GET /api/visits
- PUT /api/visits/:id

For each API include:

- Endpoint
- HTTP Method
- Request Parameters

- Request Body
- Response
- Status Codes
- Error Responses



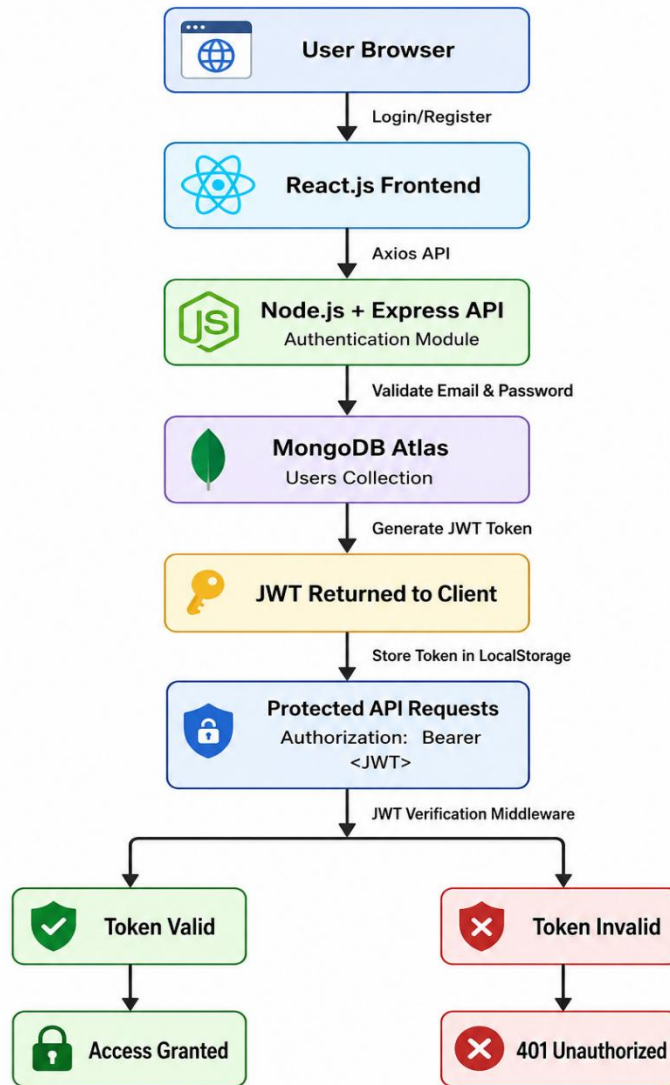
8. Authentication

8.1 Explain how authentication and authorization are handled in the project.

Authentication is implemented using JSON Web Tokens (JWT). During user registration, passwords are encrypted using bcrypt before being stored in MongoDB. Upon successful login, the server generates a JWT token that is stored in the browser's local storage.

Protected APIs require the Authorization header with the Bearer Token. Middleware verifies the token before granting access to protected resources such as adding properties, scheduling visits, or managing wishlists.

Authorization ensures that users can only modify their own properties and data.



9. User Interface

The HouseHunt application provides a clean, responsive, and user-friendly interface that allows tenants and property owners to interact with the system efficiently. The application is developed using React.js with responsive CSS to ensure compatibility across desktops, tablets, and mobile devices.

9.1 Home Page

The Home Page is the landing page of the HouseHunt application. It introduces users to the platform with an attractive interface and provides quick access to property listings, search functionality, featured properties, and navigation links. The page is designed to create a positive first impression while allowing users to easily browse available rental properties.

Features

- Responsive navigation bar
- Hero section
- Featured property listings
- Search functionality
- Footer with useful links

Find Your Dream Home

Discover apartments, villas, and houses at the best locations with HouseHunt.

[Search](#)[Browse Properties](#)[Get Started](#)

9.2 Registration Page

The Registration Page allows new users to create an account by entering their personal information such as name, email, phone number, and password. Passwords are securely encrypted before being stored in the database.

Features

- User Registration Form
- Form Validation
- Password Encryption
- Secure Account Creation
- Mobile Number Registration

Register

[Register](#)

Already have an account? [Login Here](#)

9.3 Login Page

The Login Page authenticates registered users using their email and password. After successful authentication, a JWT token is generated and stored in the browser, allowing secure access to protected resources.

Features

- Secure Login
- JWT Authentication
- Session Management
- Error Handling
- Protected Route Access

 [Home](#) [Properties](#) [Dashboard](#) [Login](#) [Register](#)

Login

Login

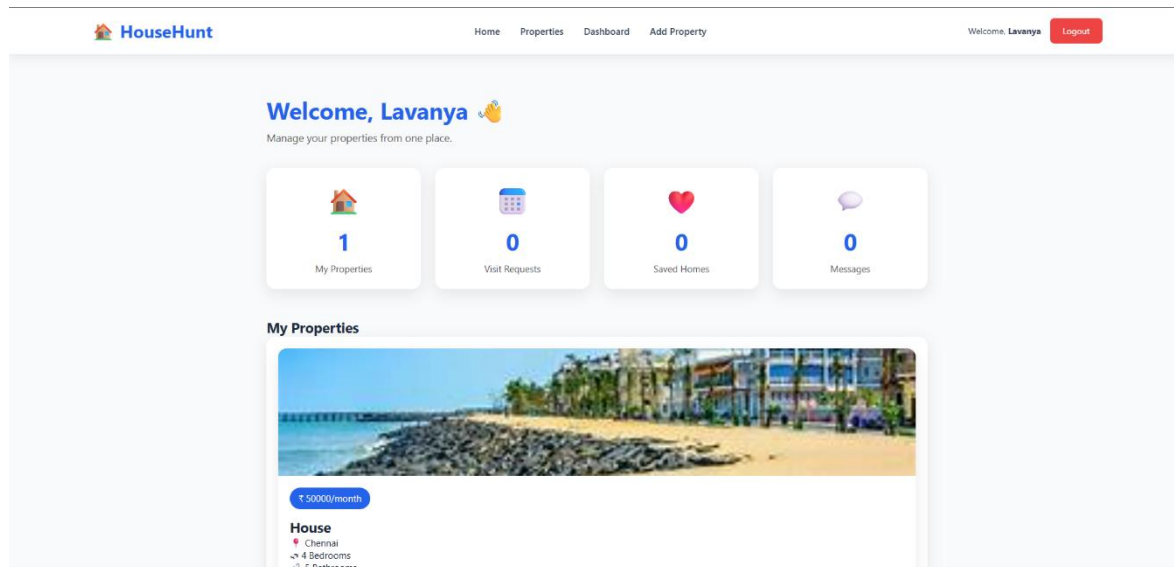
[New User? Register Here](#)

9.4 Dashboard

The Dashboard serves as the personalized workspace for property owners. It displays property statistics, visit requests, quick actions, and property management options.

Features

- Welcome Section
- Statistics Cards
- My Properties
- Visit Requests
- Quick Actions
- Property Management

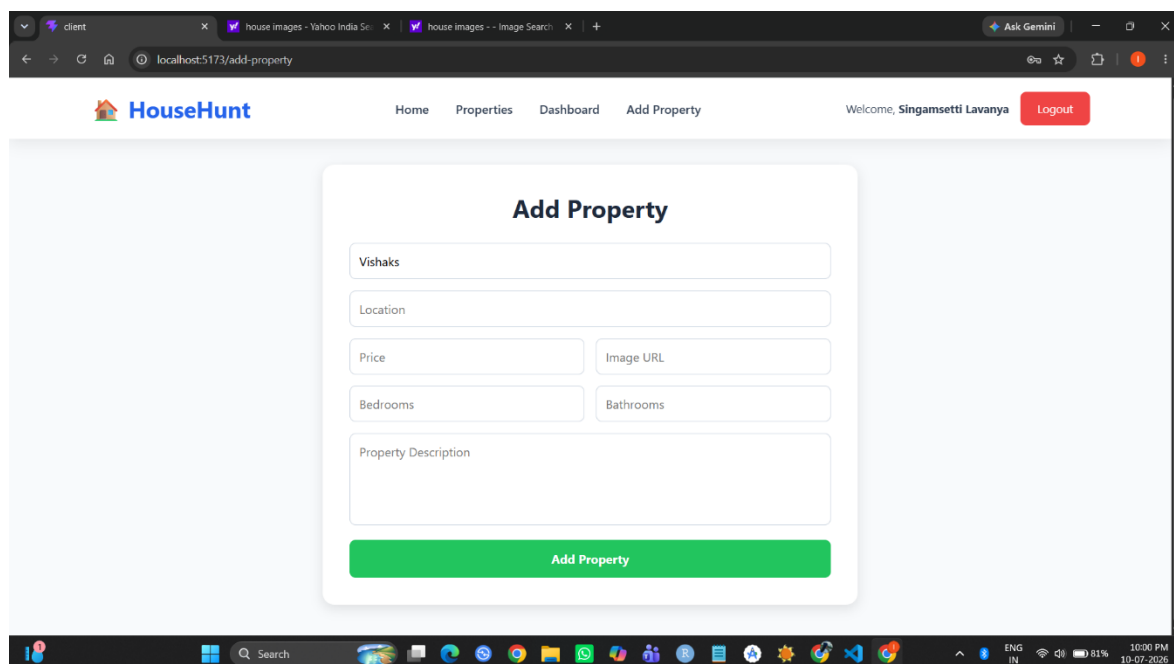


9.5 Add Property

The Add Property page enables property owners to upload new rental properties into the platform. Users can enter property details, upload images, and submit the listing.

Features

- Property Image Upload
- Property Description
- Price Information
- Bedrooms & Bathrooms
- Location Details
- Form Validation

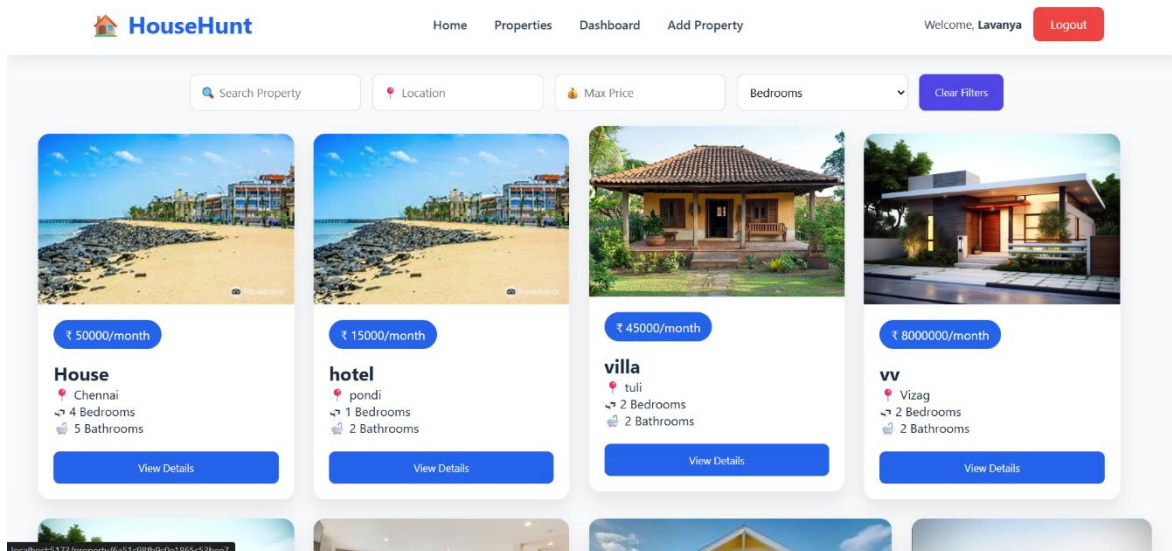


9.6 Property Listing

The Property Listing page displays all available rental properties in a well-organized card layout. Users can search and filter properties according to their preferences.

Features

- Property Cards
- Search Bar
- Responsive Grid Layout
- Property Images
- Price Display
- Location Information

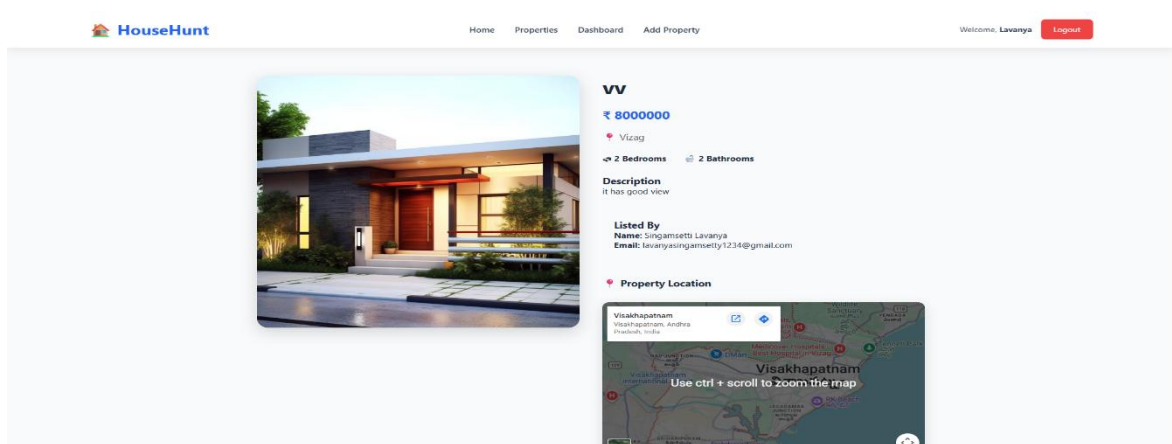


9.7 Property Details

The Property Details page provides complete information about a selected property. It includes images, owner information, description, pricing, reviews, ratings, and contact options.

Features

- Property Gallery
- Owner Information
- Property Description
- Price Details
- Bedroom & Bathroom Count
- Wishlist Button

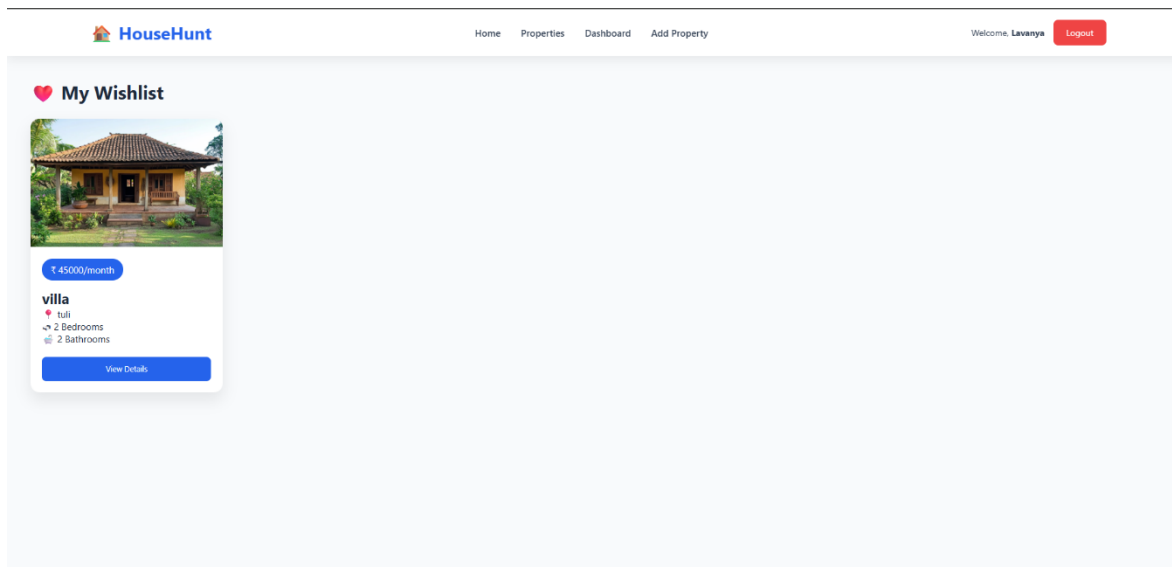


9.8 Wishlist

The Wishlist feature enables users to save their favorite properties for future reference. Users can easily access and manage their saved properties.

Features

- Save Property
- Remove Property
- Personalized Wishlist
- Quick Access

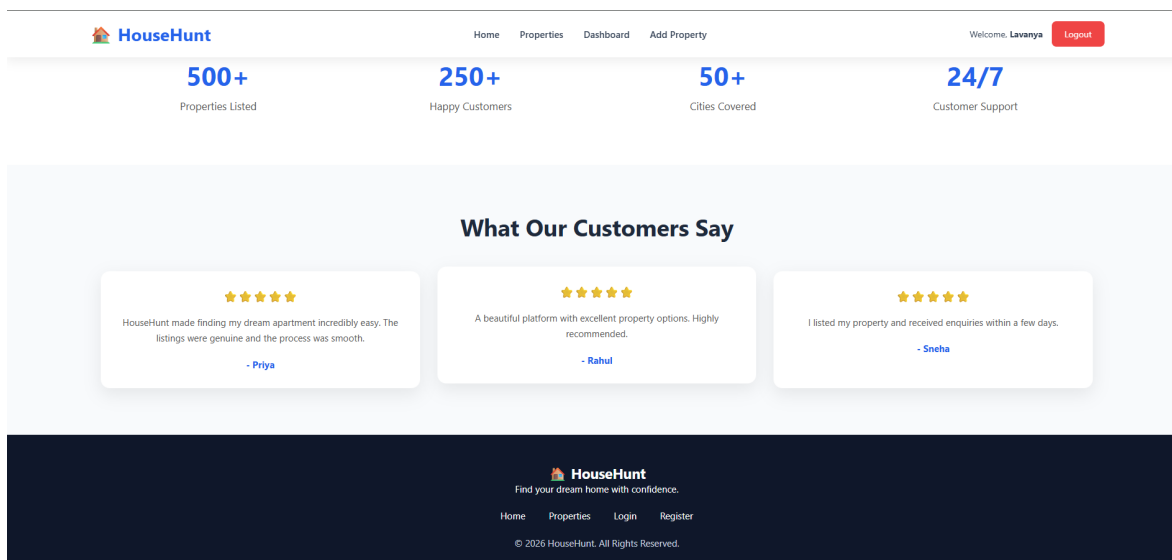


9.9 Reviews & Ratings

The Reviews section allows users to submit ratings and written reviews for properties they have visited. These reviews help future tenants make informed decisions.

Features

- Star Rating System
- User Comments
- Property Reviews
- Review Display
- Real-Time Updates



9.10 Visit Scheduling

The Visit Scheduling feature enables tenants to book appointments for property visits by selecting their preferred date and time. Property owners receive these requests through their dashboard.

Features

- Date Selection
- Time Selection
- Schedule Visit
- Visit Confirmation
- Appointment Management

The screenshot shows the 'Schedule a Visit' form on the HouseHunt website. The form includes a date selection field (dd-mm-yyyy), a time selection dropdown (Select Time), and a green 'Schedule Visit' button. Below the form are three buttons: 'Wishlist', 'Email Owner', and 'WhatsApp'. Underneath these is a 'Reviews & Ratings' section with a star rating dropdown (5 stars selected), a text area for 'Write your review...', and a 'Submit Review' button. At the bottom, there is a review from 'Ram' with a 4-star rating and the text 'nice place to hang out with family'.

9.11 Owner Dashboard

The Owner Dashboard provides property owners with complete control over their property listings and visit requests. Owners can edit listings, delete properties, and accept or reject visit requests.

Features

- Property Management
- Edit Property
- Delete Property
- Accept Visit Request
- Reject Visit Request
- View Visitor Details

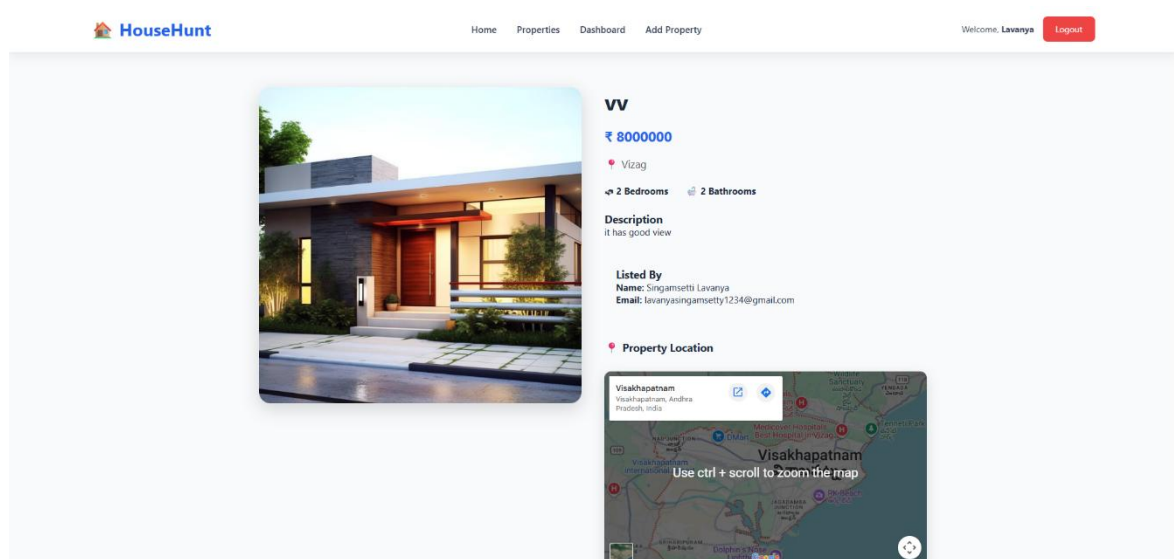
The screenshot shows the Owner Dashboard on the HouseHunt website. The dashboard is titled 'Welcome, Singamsetti Lavanya' and includes a sub-header 'Manage your properties from one place.' Below this are four summary cards: 'My Properties' (5), 'Visit Requests' (2), 'Saved Homes' (0), and 'Messages' (0). The 'My Properties' section is expanded, showing three property listings. Each listing includes a photo, a price tag (e.g., ₹ 8000000/month), and details like location (Vizag) and number of bedrooms (2, 4, and 5 respectively).

9.12 Google Maps Integration

Google Maps is integrated into the Property Details page to display the exact location of the property. This allows users to understand the surrounding area before scheduling a visit.

Features

- Embedded Google Maps
- Property Location
- Interactive Map
- Easy Navigation

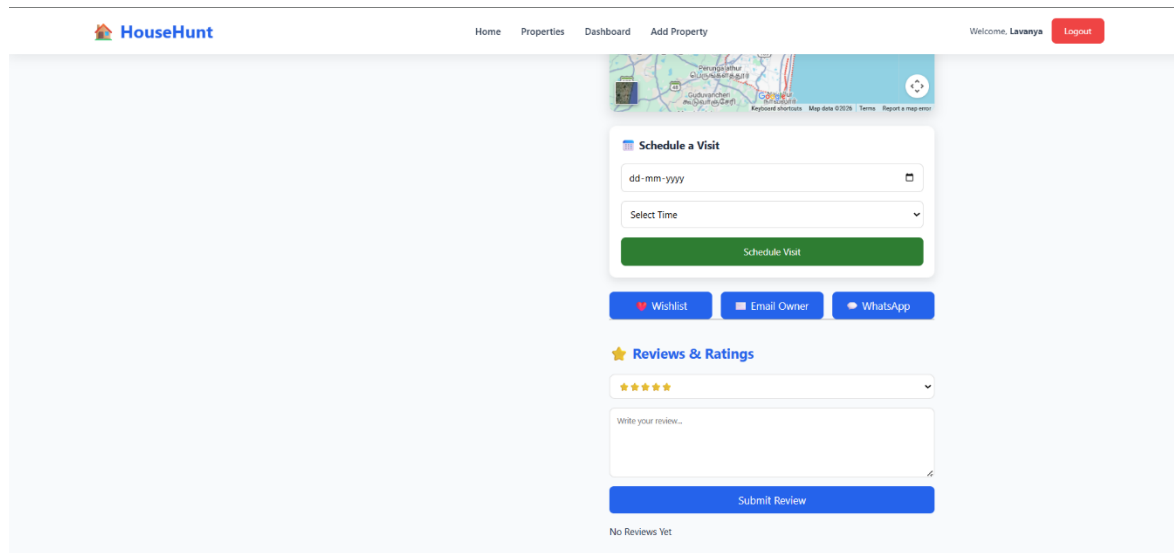


9.13 Contact Owner

The Contact Owner feature allows users to communicate directly with the property owner through Email or WhatsApp. This facilitates quick communication without involving intermediaries.

Features

- Email Integration
- WhatsApp Integration
- Owner Contact Details
- One-Click Communication



10. Testing

Testing is one of the most important phases in the software development life cycle. It ensures that the HouseHunt application functions correctly, securely, and efficiently under different scenarios. The application was tested throughout the development process to identify bugs, verify requirements, and ensure that every feature works as expected.

The testing process focused on validating both functional and non-functional requirements. Each module, including user authentication, property management, wishlist, reviews, visit scheduling, and dashboard functionalities, was tested independently as well as together to ensure seamless integration.

The testing activities included unit testing, integration testing, manual testing, API testing, authentication testing, validation testing, and browser compatibility testing. Postman was used extensively for testing backend REST APIs, while manual testing was performed on the frontend using different browsers.

10.1 Testing Objectives

The primary objectives of testing were:

- Verify that all application modules function correctly.
- Ensure secure authentication and authorization.
- Validate user inputs and prevent invalid data entry.
- Test CRUD operations for property management.
- Verify review and wishlist functionalities.
- Ensure accurate visit scheduling.
- Check API responses and status codes.
- Verify browser compatibility.
- Improve system reliability and performance.

10.2 Testing Strategy

The HouseHunt application follows a combination of manual testing and API testing. Each module was tested individually before performing end-to-end testing of the complete application.

The testing strategy includes:

- Requirement Verification
- Module Testing
- Integration Testing
- API Testing
- User Acceptance Testing
- Bug Fixing
- Regression Testing

Tool	Purpose
Postman	API Testing
Google Chrome	UI Testing
Visual Studio Code	Debugging
MongoDB Atlas	Database Verification
React Developer Tools	Frontend Debugging
Node.js Console	Backend Logging
Browser Developer Tools	Network & Console Inspection

10.4 Types of Testing Performed

1. Unit Testing

Each backend controller and frontend component was tested individually to verify its functionality.

Examples include:

- User Registration Controller
- Login Controller
- Property Controller
- Wishlist Controller
- Review Controller
- Visit Controller

Result:

Each function performs the intended operation without affecting other modules.

2. Integration Testing

Integration testing was performed to ensure proper communication between:

- React Frontend
- Express Backend
- MongoDB Database

The testing verified that data entered through the frontend is correctly stored and retrieved from MongoDB using REST APIs.

3. Functional Testing

Each application feature was tested against the functional requirements.

Examples:

- User Registration
- Login
- Add Property
- Edit Property
- Delete Property
- Wishlist
- Reviews
- Visit Scheduling

All expected outputs matched the system requirements.

4. Manual Testing

Manual testing was conducted by navigating through every page of the application and verifying that each button, form, and feature worked correctly.

The following scenarios were tested:

- Invalid Login
- Empty Registration Form
- Duplicate Registration
- Property Search
- Wishlist Addition
- Visit Scheduling
- Accept/Reject Visit Request

5. API Testing Using Postman

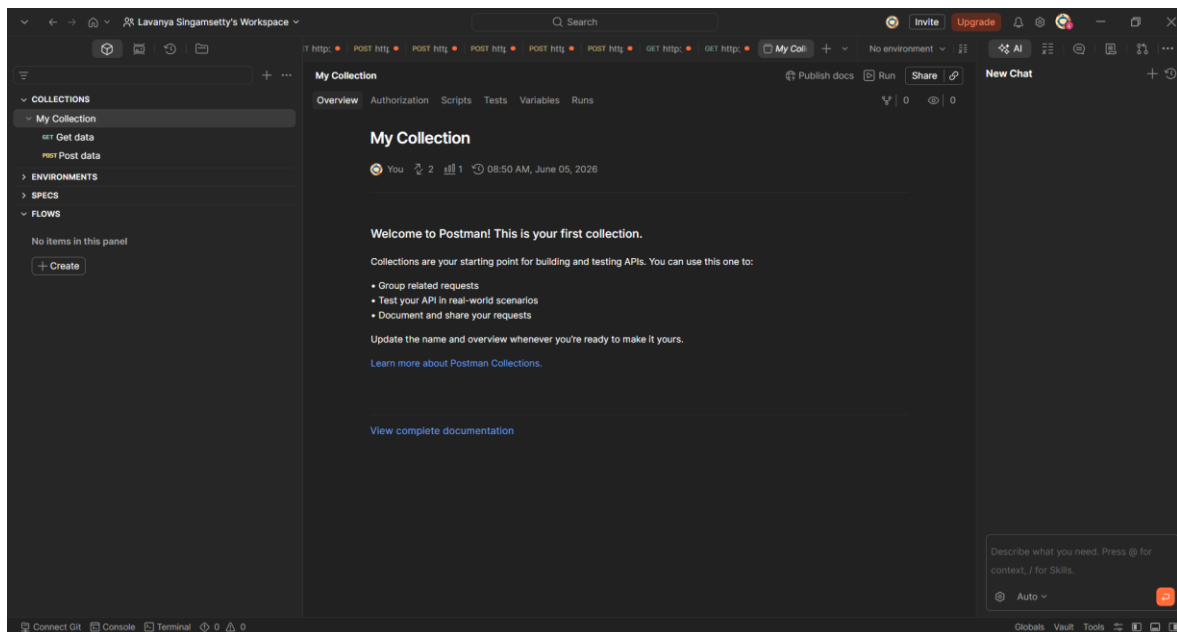
Postman was used to test all backend REST APIs independently before integrating them with the frontend.

The following APIs were tested:

- User Registration
- User Login
- Get Properties
- Add Property
- Update Property
- Delete Property
- Wishlist
- Reviews
- Visit Scheduling
- Accept Visit
- Reject Visit

Each API was verified for:

- Request Validation
- Response Body
- Response Time
- Status Code
- Authorization
- Error Handling



6. Authentication Testing

JWT authentication was tested by:

- Logging in with valid credentials
- Logging in with invalid credentials
- Accessing protected APIs without token
- Accessing protected APIs using valid token
- Token expiration validation

Expected Result:

- Unauthorized users receive **401 Unauthorized**
- Authorized users receive successful responses

7. Validation Testing

Validation testing ensured that invalid data could not be submitted.

Examples include:

- Empty Fields
- Invalid Email Format

- Weak Password
- Missing Property Details
- Missing Visit Date
- Missing Review

Toast notifications displayed appropriate validation messages.

8. Browser Compatibility Testing

The application was tested on:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

The UI remained responsive across all supported browsers.

10.5 Test Cases Executed

The following functionalities were successfully tested:

- User Registration
- User Login
- Property Listing
- Property Search
- Property Details
- Property CRUD Operations
- Wishlist
- Reviews
- Visit Scheduling
- Dashboard
- Google Maps Integration
- Contact Owner
- Accept Visit Request
- Reject Visit Request

10.6 API Testing Using Postman

Step 1: Open Postman

Launch the Postman application.

Step 2: Start Backend Server

```
cd server
npm run dev
```

Ensure the backend is running at: <http://localhost:5000>

Step 3: Test User Registration

Method

POST

URL - <http://localhost:5000/api/auth/register>

Body → Raw → JSON

```
{
  "name": "Lavanya",
```

```
"email": "lavanya@gmail.com",
```

```
"phone": "9876543210",
```

```
"password": "123456"
```

```
}
```

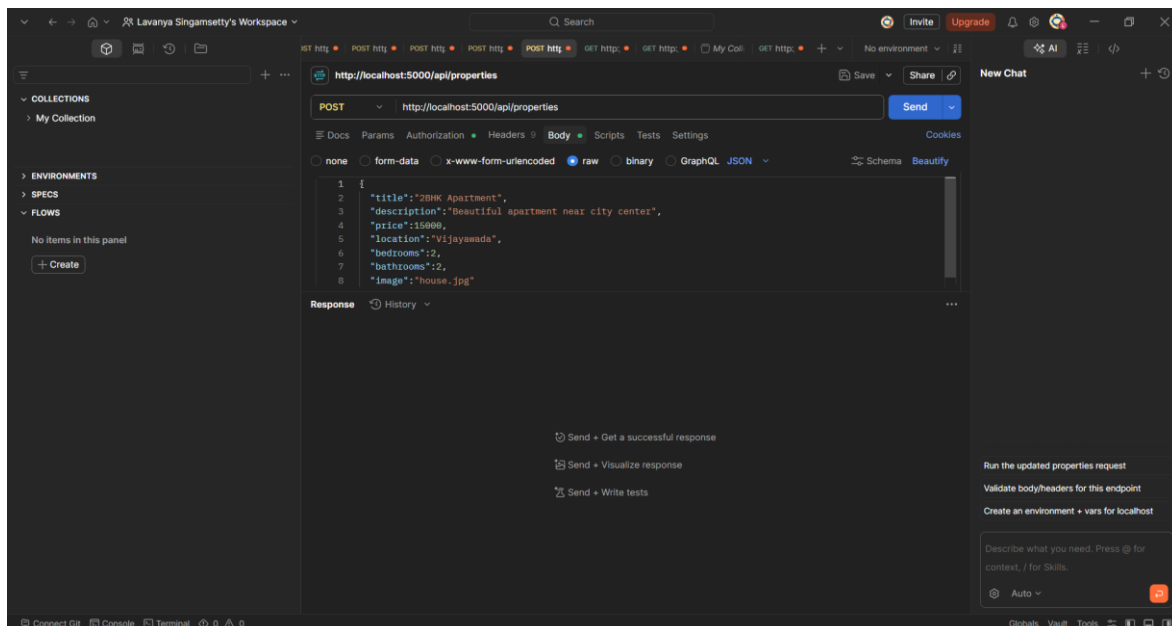
Click **Send**

Expected Response

```
{
```

```
"message": "User Registered Successfully"
```

```
}
```



Step 4: Test Login

Method

POST

URL - <http://localhost:5000/api/auth/login>

Body

```
{
```

```
"email": "lavanya@gmail.com",
```

```
"password": "123456"
```

```
}
```

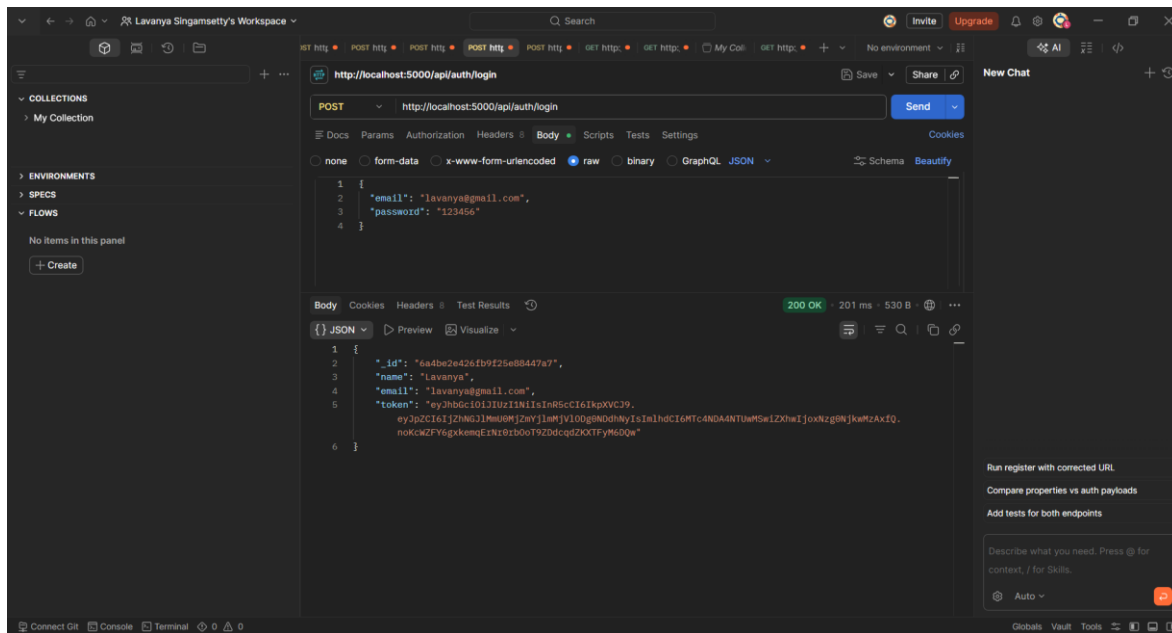
Response

```
{
```

```
"token": "eyJhbGciOiJIUzI1NiIs..."
```

```
}
}
```

Copy the JWT Token.



Step 5: Add Authorization Token

Go to

Authorization

Select

Bearer Token

Paste

JWT Token

Now protected APIs can be accessed.

Step 6: Test Add Property API

Method

POST

URL - <http://localhost:5000/api/properties>

Authorization

Bearer Token

Body → Form-data

Title

Description

Price

Location

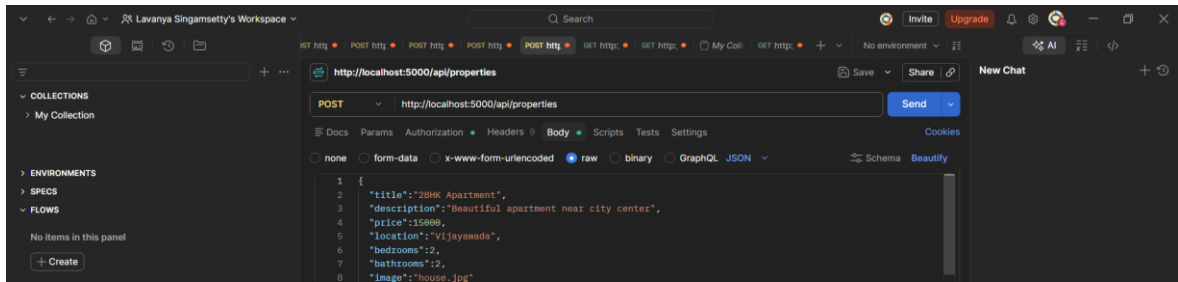
Bedrooms

Bathrooms

image

Click - Send

Expected Result - Property Added Successfully



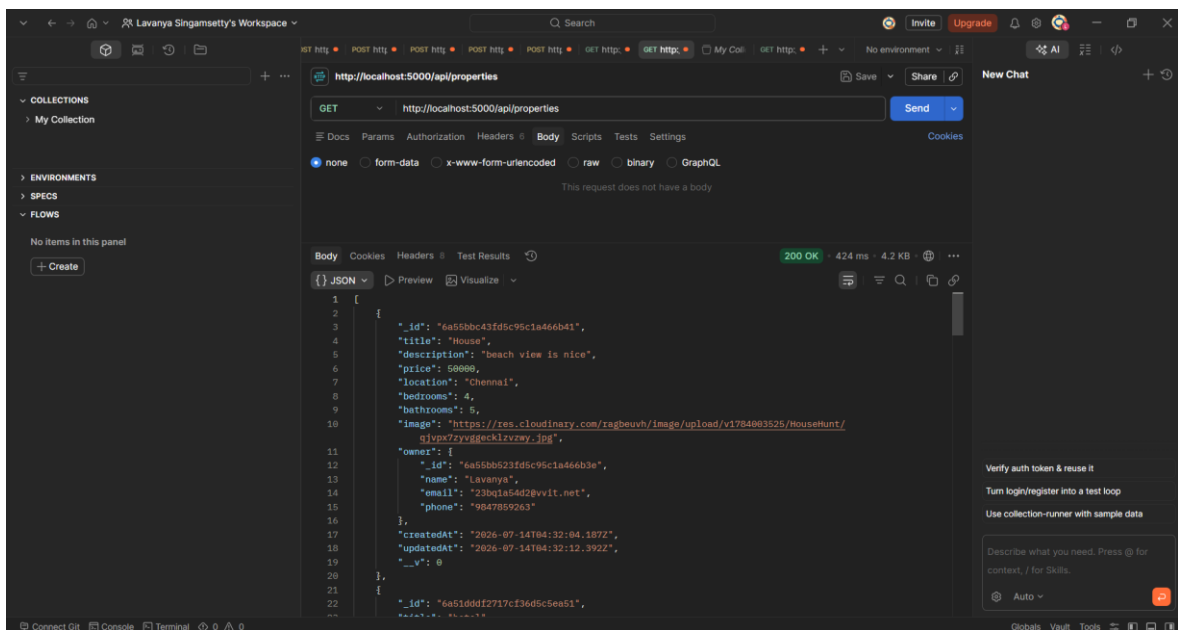
Step 7: Test Get Properties

Method

GET

URL - <http://localhost:5000/api/properties>

Expected Result - List of Properties



Step 8: Test Wishlist

Method

POST

URL - <http://localhost:5000/api/wishlist>

Body

```

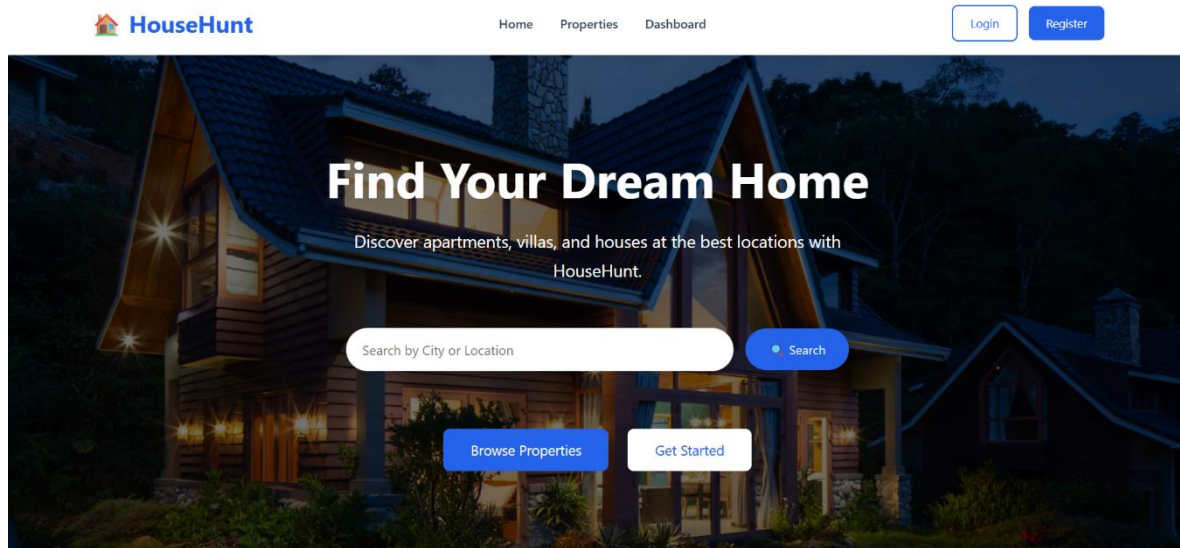
{
  "propertyId": "PROPERTY_ID"
}

```

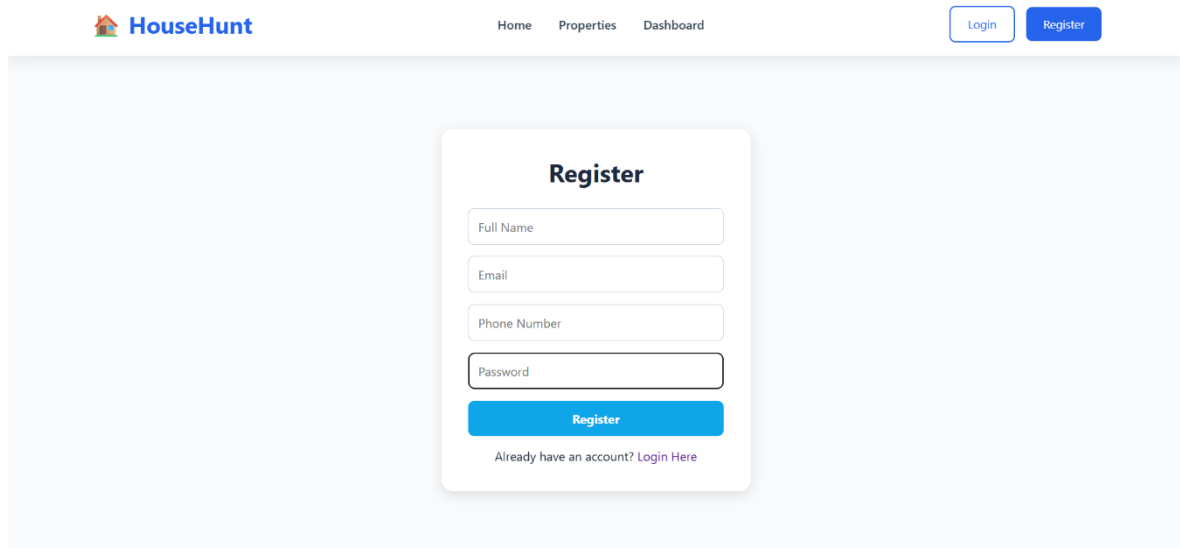
Expected Result - Added to Wishlist

11.Screenshots or Demo

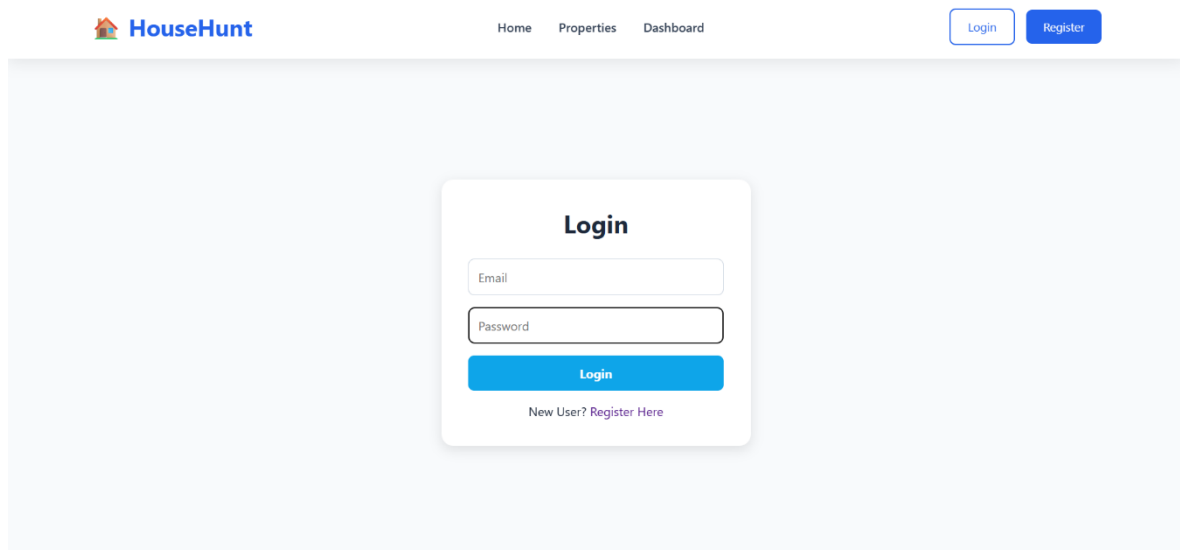
Home :



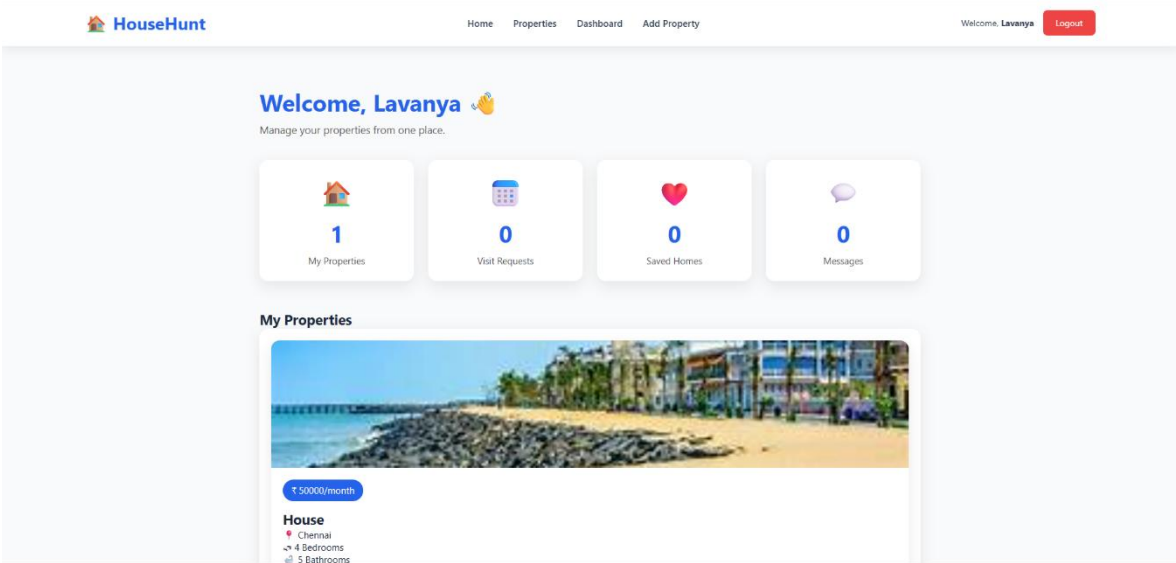
Registration :



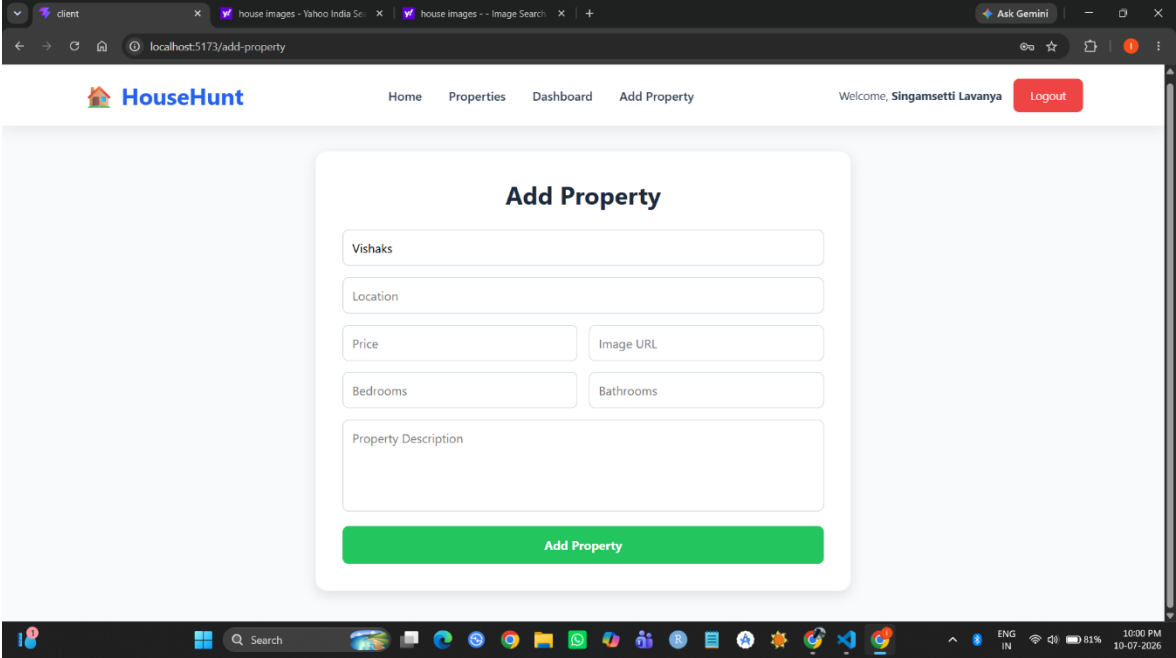
Login :



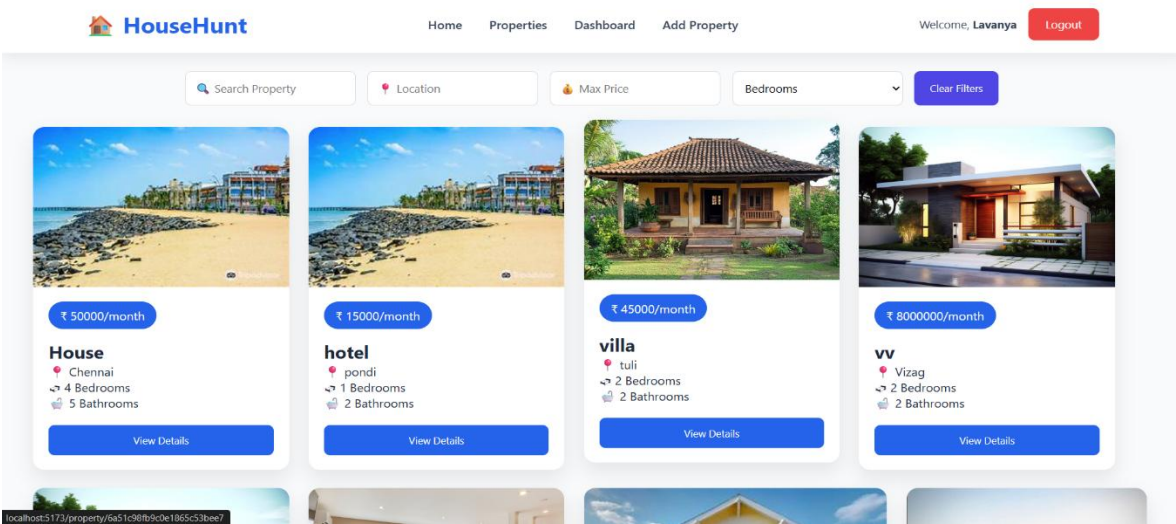
DashBoard :



Add Property:



Property Listing:



Property Details:



VV

₹ 8000000

Vizag

2 Bedrooms 2 Bathrooms

Description
It has good view

Listed By
Name: Singamsetti Lavanya
Email: lavanyasingamsetty1234@gmail.com

Property Location

Visakhapatnam

Visakhapatnam, Andhra Pradesh, India

Use ctrl + scroll to zoom the map

Google Maps:



VV

₹ 8000000

Vizag

2 Bedrooms 2 Bathrooms

Description
It has good view

Listed By
Name: Singamsetti Lavanya
Email: lavanyasingamsetty1234@gmail.com

Property Location

Visakhapatnam

Visakhapatnam, Andhra Pradesh, India

Use ctrl + scroll to zoom the map

WishList:



My Wishlist

₹ 45000/month

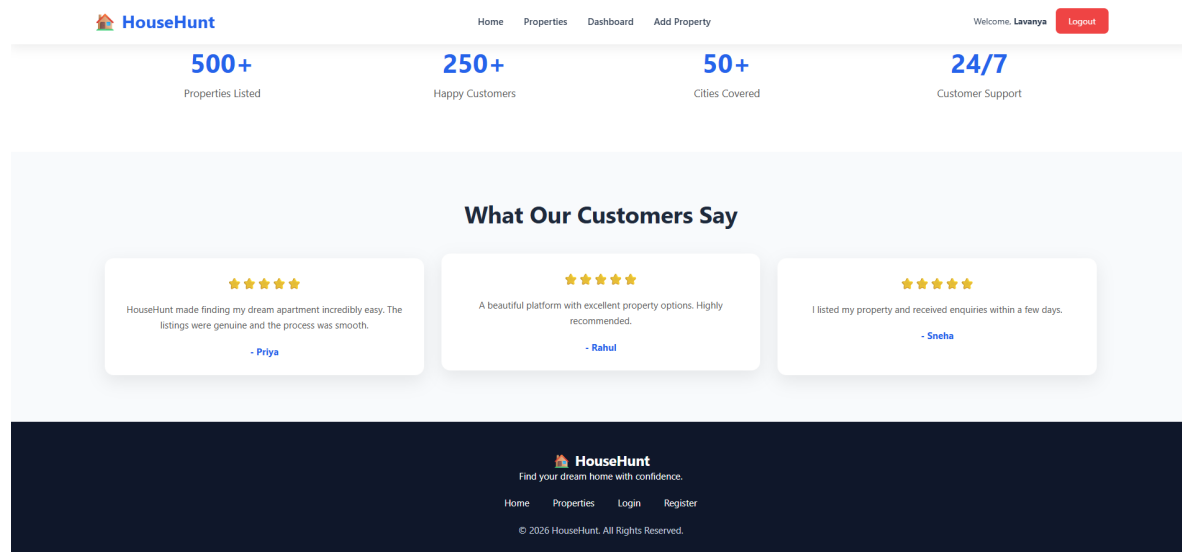
villa

tuli

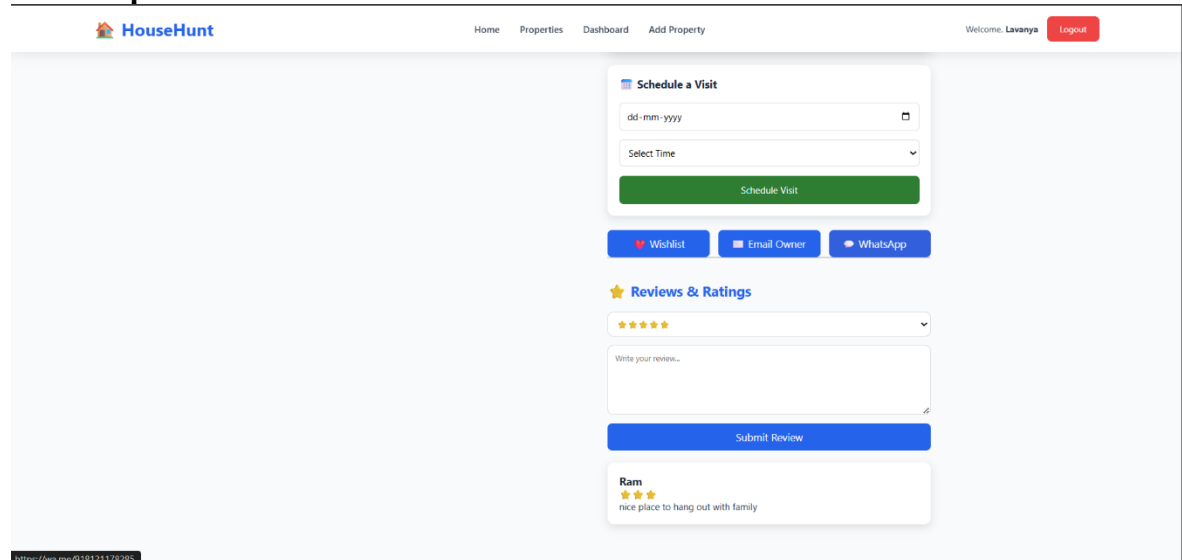
2 Bedrooms 2 Bathrooms

View Details

Reviews:



Visit Requests:



12. Known Issues

12.1 Document any known bugs or issues that users or developers should be aware of.

- Property images are currently stored locally rather than in cloud storage.
- No online payment integration.
- Email notifications are not automated.
- Push notifications are not available.
- AI-based property recommendations are not implemented.
- Advanced analytics dashboard is not available.
- Chat functionality between tenants and owners is not implemented.

13.Future Enhancements

13.1 Outline potential future features or improvements that could be made to the project.

- Future enhancements planned for HouseHunt include:
- AI-Based Property Recommendation System
- Property Price Prediction using Machine Learning
- Cloudinary Image Storage
- Online Payment Gateway Integration
- Google OAuth and Social Login
- Real-Time Chat using Socket.IO
- Push Notifications
- Mobile Application using React Native
- Virtual Property Tours
- Property Comparison Feature
- Advanced Analytics Dashboard
- Admin Panel with Reports
- Voice Search Functionality
- Multi-language Support
- Cloud Deployment using AWS